

# Shipping on the Air

Assignment #03 - Software Architecture and Platforms

A.Y. 2025/2026

Fabio Vincenzi

20 agosto 2026

## Indice

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                             | <b>2</b>  |
| <b>2</b> | <b>Event-driven Architecture</b>                | <b>2</b>  |
| 2.1      | Scelta del candidato . . . . .                  | 2         |
| 2.2      | Da orchestrazione a coreografia . . . . .       | 2         |
| 2.3      | Lo strato dei canali . . . . .                  | 3         |
| 2.4      | Request/reply su un bus di eventi . . . . .     | 5         |
| 2.5      | Il confine dell'architettura a eventi . . . . . | 5         |
| 2.6      | Comandi ed eventi . . . . .                     | 7         |
| <b>3</b> | <b>Scaling e Service Levels</b>                 | <b>7</b>  |
| 3.1      | SLO e SLI . . . . .                             | 7         |
| 3.2      | Strumentazione . . . . .                        | 8         |
| 3.3      | Verifica degli indicatori . . . . .             | 9         |
| 3.4      | Deployment su Kubernetes . . . . .              | 9         |
| 3.5      | Replicazione . . . . .                          | 10        |
| 3.6      | Self-healing osservato . . . . .                | 11        |
| <b>4</b> | <b>Impatto sui test</b>                         | <b>11</b> |
| <b>5</b> | <b>Limiti noti</b>                              | <b>12</b> |
| <b>6</b> | <b>Conclusioni</b>                              | <b>13</b> |

# 1 Introduzione

Questo documento descrive come il sistema *Shipping on the Air*, raffinato con i micro-services patterns nell'Assignment 2, sia stato ulteriormente evoluto lungo due direzioni: l'adozione di un'architettura **event-driven** per uno dei microservizi, con Apache Kafka come middleware, e la definizione di **service level** misurabili accompagnata da un deployment su **Kubernetes**.

Il dominio non cambia. Restano **order-service**, **delivery-service**, **drone-service** e l'**api-gateway**. Cambia il modo in cui i servizi si coordinano, il modo in cui il sistema viene misurato e la piattaforma su cui viene eseguito.

## 2 Event-driven Architecture

### 2.1 Scelta del candidato

La scelta è caduta sul **delivery-service**, per tre ragioni.

La prima è di posizione: il **delivery-service** è il nodo centrale del sistema. Riceve una richiesta dall'**order-service** quando un ordine viene confermato, si rivolge al **drone-service** per ottenere un mezzo, e informa entrambi quando la consegna termina. È quindi il punto in cui l'accoppiamento sincrono produce il danno maggiore, e dove eliminarlo produce il beneficio maggiore.

La seconda è di preparazione. Nell'Assignment 2 il servizio era già stato riscritto con **Event Sourcing**: gli eventi di dominio esistono già come concetti di prima classe, l'aggregate è già diviso fra una fase **process** che decide e una **apply** che muta lo stato, e l'event store è già nascosto dietro una porta. Adottare un'architettura a eventi non richiede quindi di inventare gli eventi, ma soltanto di farli uscire dal processo.

La terza è di coerenza concettuale. Event Sourcing ed Event-driven Architecture condividono l'unità di base, il fatto avvenuto, ma la applicano a problemi diversi: il primo alla persistenza, il secondo alla comunicazione. Applicarli allo stesso servizio rende esplicito questo rapporto, che la sezione 2.6 riprende.

### 2.2 Da orchestrazione a coreografia

Nell'Assignment 2 la creazione di una consegna era **orchestrata**: un componente conosceva l'intera sequenza e la eseguiva chiamando gli altri.

Listing 1: Assignment 2: il servizio chiama il drone-service e attende

```
var droneId = droneService.requestAvailableDrone(pickupLat ,
    pickupLng , weightKg);
if (droneId.isPresent()) {
    delivery.assignDrone(droneId.get());
    repository.save(delivery);
}
```

Questo codice non esiste più. Al suo posto il servizio registra il fatto e prosegue; il **drone-service** se ne accorge e risponde di propria iniziativa. Il coordinamento non è più scritto in un punto, ma emerge dalle reazioni: è una **coreografia**.

La conseguenza si legge in una sola riga, il costruttore dell'application service:

Listing 2: La firma che riassume l'intero punto 1

```
// Assignment 2
public DeliveryServiceImpl(DeliveryRepository repository,
                           DroneServicePort droneService,
                           OrderServicePort orderService)

// Assignment 3
public DeliveryServiceImpl(DeliveryRepository repository)
```

Il delivery-service non conosce più l'esistenza degli altri servizi. Prima li dichiarava come dipendenze; ora sa soltanto produrre fatti. Al loro posto è comparso il metodo `assignDrone(DeliveryId, String)`, che è la porta d'ingresso per la risposta del drone-service.

La sequenza completa è riportata nella tabella 1.

Tabella 1: La coreografia della creazione di una consegna

| # | Attore           | Azione                                                                                                                                |
|---|------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| 1 | utente           | POST /api/v1/orders/{id}/confirm sull'api-gateway                                                                                     |
| 2 | order-service    | conferma l'ordine e annuncia il fatto su <code>order-confirmed</code> , senza rivolgersi a nessuno in particolare                     |
| 3 | delivery-service | reagisce al fatto creando la consegna in stato <code>SCHEDULED</code> , e ne annuncia la nascita su <code>new-delivery-created</code> |
| 4 | drone-service    | reagisce all'annuncio e pubblica <code>drone-assigned</code> oppure <code>drone-unavailable</code>                                    |
| 5 | delivery-service | riceve l'assegnazione e la consegna passa a <code>DRONE_ASSIGNED</code>                                                               |

Il passo 2 è deliberato: confermare un ordine non attende che la consegna esista. Ne consegue che il tracking mostra per un breve intervallo lo stato `SCHEDULED` con `droneId` nullo, e solo successivamente `DRONE_ASSIGNED`. Sul deployment Kubernetes la transizione è stata osservata in **2 secondi**.

Questo punto merita attenzione, perché rappresenta un cambiamento di significato e non solo di implementazione. Nell'Assignment 2 lo stato “consegna programmata ma senza drone” era descritto dal QAS di availability come il **comportamento degradato** atteso in caso di guasto del drone-service. Nell'Assignment 3 lo stesso stato è il **comportamento normale**, attraversato da ogni consegna. La consistenza da immediata è diventata *eventual*, ed è il prezzo strutturale dell'architettura a eventi.

## 2.3 Lo strato dei canali

Un *event channel* corrisponde a un topic Kafka omonimo. Nel modulo `common` sono state introdotte due classi.

Tabella 2: Le classi del modulo `common`

| Classe                          | Responsabilità                                                                   |
|---------------------------------|----------------------------------------------------------------------------------|
| <code>OutputEventChannel</code> | pubblica un <code>JsonObject</code> , incapsulando un <code>KafkaProducer</code> |
| <code>InputEventChannel</code>  | sottoscrive un canale e invoca un handler per ogni evento                        |

**La chiave di partizione.** Un topic Kafka è diviso in partizioni e l'ordinamento è garantito solo *all'interno* di una partizione; i record privi di chiave vengono distribuiti su tutte. Senza chiave, due eventi relativi alla stessa consegna potrebbero quindi essere consegnati in ordine invertito. Passando l'identificatore dell'entità come chiave si ottiene che tutti i suoi eventi ricadano sulla stessa partizione.

Listing 3: La chiave rende l'ordinamento una proprietà per entità

```
public Future<Void> postEvent(String key, JsonObject event) {
    event.put("timestamp", System.currentTimeMillis());
    var record = KafkaProducerRecord.<String, String>create(name,
        key, event.encode());
```

**Il consumer group.** È il parametro che distingue due comportamenti opposti espressi dallo stesso meccanismo. Kafka non elimina un messaggio quando viene letto: mantiene un *offset*, cioè un segnalibro, per ogni combinazione di gruppo, topic e partizione, e all'interno di un gruppo ciascuna partizione ha un solo proprietario. Ne segue che consumatori in gruppi **diversi** ricevono ognuno l'intero flusso, mentre consumatori nello **stesso** gruppo se lo dividono in parti disgiunte: dentro un gruppo nessun messaggio è mai consegnato due volte.

Entrambe le regole sono state verificate sul broker del cluster, pubblicando tre messaggi su un topic con una sola partizione e rileggendolo cinque volte al variare del gruppo.

Tabella 3: Verifica sperimentale della semantica dei gruppi, tre messaggi su una partizione

| Lettore                   | Ricevuti | Motivo                                          |
|---------------------------|----------|-------------------------------------------------|
| gruppo A, prima lettura   | 3        | segnalibro a zero, legge tutto                  |
| gruppo A, seconda lettura | 0        | il suo segnalibro è ormai in fondo              |
| gruppo B, prima lettura   | 3        | segnalibro proprio, indipendente da quello di A |
| gruppo C, consumatore 1   | 3        | è il proprietario della partizione              |
| gruppo C, consumatore 2   | 0        | stessa partizione, ma non gli appartiene        |

Che il gruppo B rilegga tutto dimostra che la lettura **non consuma**: ad avanzare era stata solo la posizione di A. E il secondo consumatore del gruppo C non riceve nulla non perché i messaggi siano stati distrutti o sottratti dal collega, ma perché l'unica partizione era già assegnata all'altro membro del suo gruppo.

Da qui due conseguenze. Ogni servizio usa un **gruppo distinto**: condividerlo produrrebbe un guasto parziale e silenzioso, con ciascuno dei due che vede solo metà degli eventi di interesse comune. E il numero di partizioni è il **tetto ai consumatori utilmente attivi** di un gruppo: la capacità si aumenta ripartizionando il topic, non moltiplicando i consumatori.

Infine, i consumatori sono configurati con `auto.offset.reset`: `earliest`: un gruppo privo di segnalibro parte dall'inizio del log anziché dal presente. È la proprietà per cui un servizio riavviato recupera da sé gli eventi persi durante l'assenza.

## 2.4 Request/reply su un bus di eventi

Un canale di eventi non possiede il concetto di risposta, che HTTP fornisce nativamente. Il pattern adottato associa a ogni operazione **tre canali**: uno di richiesta e due di esito, che trasportano l'informazione altrimenti veicolata dal codice di stato.

Listing 4: La terna di canali per l'interrogazione di una consegna

|                                             |                                                 |
|---------------------------------------------|-------------------------------------------------|
| <code>get-delivery-requests</code>          | <code>richiesta</code>                          |
| <code>get-delivery-requests-approved</code> | <code>esito positivo</code>                     |
| <code>get-delivery-requests-rejected</code> | <code>esito negativo, con la motivazione</code> |

Il delivery-service espone una sola terna, quella dell'interrogazione: è l'unico punto del sistema in cui qualcuno pone una domanda e ne attende l'esito. Tutte le altre interazioni sono fatti annunciati, come la sezione successiva descrive.

I canali di esito sono però condivisi da tutti i chiamanti, quindi ogni richiesta porta con sé un `requestId` che la risposta ripete. Nell'api-gateway questo consente di ricostruire una chiamata sincrona a partire da uno scambio asincrono: il proxy registra una risposta pendente sotto l'identificatore, pubblica la richiesta e attende.

Listing 5: Correlazione richiesta-risposta nell'api-gateway

```
private final Map<String, CompletableFuture<JsonObject>> pending
    = new ConcurrentHashMap<>();

private void complete(JsonObject reply) {
    var waiting = pending.remove(reply.getString("requestId"));
    if (waiting != null) {
        waiting.complete(reply);
    }
}
```

L'attesa è limitata da un timeout di 5 secondi, oltre il quale viene sollevata `ServiceNotAvailableException`; il blocco è ammissibile perché il controller invoca la porta all'interno di `executeBlocking`, quindi fuori dall'event loop. Va osservato che una *rejection* non è un errore del sistema: è il modo in cui l'informazione "nessuna consegna per questo ordine" viaggia su un canale, e viene ritradata in un `Optional.empty()`.

**Validazione.** Il controllo che con HTTP faceva il `BodyHandler` qui non esiste: un evento è JSON privo di schema. Due validatori espliciti lo intercettano prima, producendo un rifiuto leggibile come `missing or not numeric field: pickupLat`.

## 2.5 Il confine dell'architettura a eventi

La consegna consente esplicitamente agli altri microservizi di mantenere l'architettura originale, adattando solo quanto serve a interagire con quello riprogettato. Il confine è stato tracciato come segue.

Tabella 4: Cosa cambia per ciascun servizio

| Servizio         | Adattamento                                                                                                            |
|------------------|------------------------------------------------------------------------------------------------------------------------|
| delivery-service | interamente a eventi. Dell'API REST resta il solo <code>/health</code>                                                 |
| api-gateway      | mantiene l'API REST verso l'esterno e i proxy HTTP verso order e drone; solo quello verso il delivery diventa a canali |
| order-service    | mantiene la propria architettura. Annuncia <code>order-confirmed</code> attraverso un observer                         |
| drone-service    | mantiene la propria architettura. Reagisce a <code>new-delivery-created</code> e riferisce le posizioni                |

Il mantenimento di `/health` è necessario, perché è il punto su cui si appoggiano le probe di Kubernetes.

L'adattamento degli altri servizi ha richiesto la sola sostituzione degli adapter in uscita: le porte già dichiarate dai rispettivi domini, come `DeliveryServicePort` nel drone-service, sono rimaste invariate, e ne è stata fornita un'implementazione che pubblica su un canale invece di effettuare una chiamata HTTP. Nessuna logica di dominio è stata toccata, il che è il beneficio che l'architettura esagonale, adottata nell'Assignment 1, promette proprio per casi come questo.

La figura 1 riassume il risultato: gli archi continui sono chiamate REST sincrone, quelli tratteggiati canali di eventi. Il delivery-service è l'unico servizio che nessuno raggiunge più in modo sincrono.

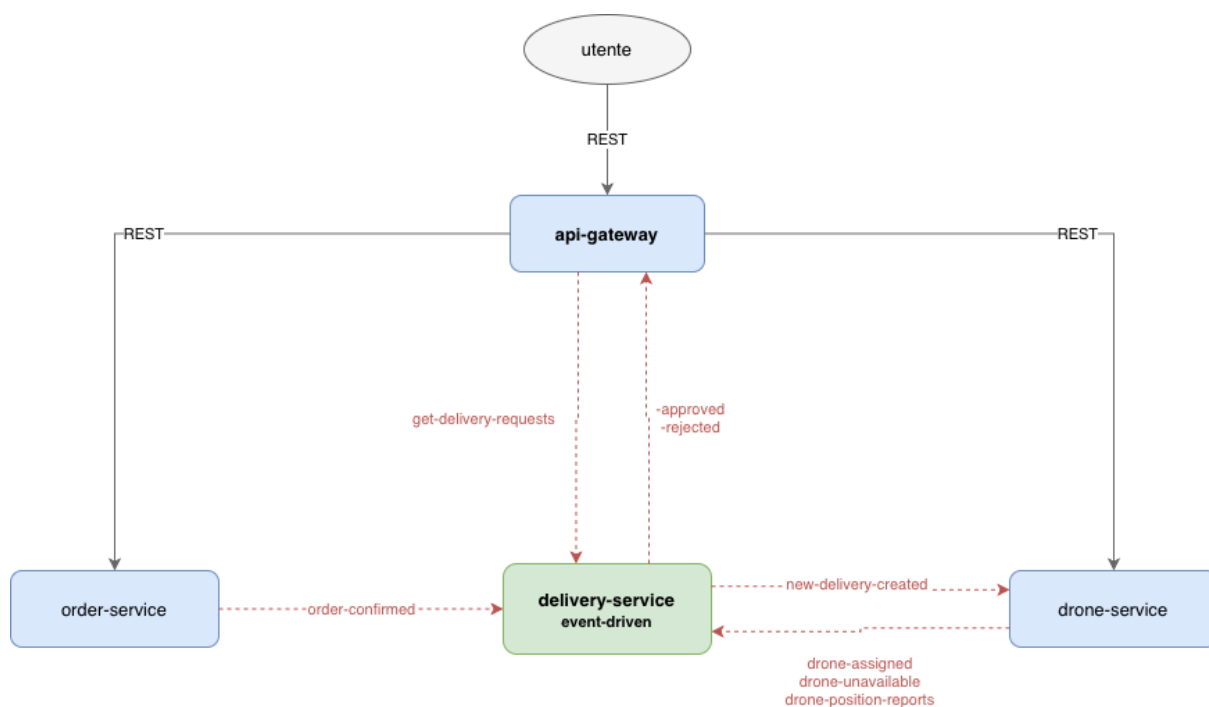


Figura 1: Canali di eventi e chiamate REST fra i servizi

## 2.6 Comandi ed eventi

Non tutto ciò che viaggia sui canali ha la stessa natura, e la distinzione ha guidato la forma degli adapter in uscita.

Un **comando** è rivolto a un destinatario preciso e attende un esito: chi lo invia decide che cosa deve accadere. È il caso dell'interrogazione che l'api-gateway rivolge al delivery-service per costruire il tracking, e infatti lì esiste una porta, `DeliveryServicePort`, dichiarata dal livello applicativo del gateway e implementata da un proxy.

Un **evento** constata un fatto avvenuto, non ha destinatario e non attende nulla: chi lo riceve decide che cosa farne. È il caso dell'ordine confermato e della posizione di un drone. Escono attraverso gli **observer**, lo stesso meccanismo già usato dall'Assignment 2 per l'osservabilità, con una seconda implementazione che pubblica su un canale.

Listing 6: Il servizio annuncia, senza sapere chi ascolta

```
observers.forEach(o -> o.notifyOrderConfirmed(event));
```

Ne discende una proprietà verificabile del sistema: nell'intero progetto **esiste una sola porta** verso il delivery-service, quella dell'api-gateway, ed esiste perché il gateway è l'unico che pone una domanda. Tutto il resto sono fatti annunciati.

Il beneficio pratico è che aggiungere un consumatore non costa nulla a chi pubblica: un servizio interessato agli ordini confermati si iscrive al canale, e né l'order-service né il suo dominio vengono toccati. Con un comando avremmo dovuto aggiungere una chiamata al mittente.

**Rapporto con l'Event Sourcing.** Gli eventi che il dominio del delivery-service emette restano **interni**: sono la sua fonte di verità, vengono persistiti nell'event store e alimentano il read model attraverso il proiettore.

Listing 7: Il read model e' derivato dallo stream, e ricostruibile da esso

```
newEvents.forEach(event -> projector.project(delivery.getId(),  
event));
```

Ciò che viaggia sui canali è invece un **evento di integrazione**: lo stesso fatto, riformulato per chi lo riceve. L'evento di dominio `DeliveryScheduled` porta l'intera `Route` e l'istante di creazione, perché deve bastare a ricostruire l'aggregate nel replay; l'annuncio `new-delivery-created` porta soltanto punto di ritiro e peso, che sono i due dati su cui il drone-service decide se ha un mezzo adatto.

La distinzione è deliberata: pubblicare gli eventi di dominio così come sono renderebbe il modello interno un **contratto pubblico**, e riorganizzare l'aggregate diventerebbe una modifica da concordare con tutti i consumatori. I due rispondono del resto a criteri diversi: un evento di dominio deve contenere quanto basta a *ricostruire lo stato*, un evento di integrazione quanto basta ad *agire senza richiamare il mittente*.

## 3 Scaling e Service Levels

### 3.1 SLO e SLI

Gli indicatori tipici con cui si esprime un livello di servizio sono disponibilità, latenza, tasso di errore, throughput e durabilità. La scelta è caduta su **disponibilità** e **throughput**, i

due che si agganciano agli scenari di qualità già formulati nell'Assignment 1: il primo alla continuità del servizio in presenza di guasti, il secondo alla scalabilità sotto un volume crescente di ordini. Entrambi sono definiti sull'**api-gateway**, unico componente esposto all'esterno e quindi il punto di osservazione più vicino all'esperienza d'uso.

Tabella 5: Gli obiettivi e i rispettivi indicatori

|              |                                                                   |
|--------------|-------------------------------------------------------------------|
| <b>SLO 1</b> | almeno il 99,5% delle richieste servite con successo su 30 giorni |
| <b>SLI 1</b> | percentuale di richieste con status inferiore a 400 sul totale    |
| <b>SLO 2</b> | piu' di 40 richieste al secondo sostenute su 30 giorni            |
| <b>SLI 2</b> | numero di richieste servite al secondo                            |

Un 4xx è considerato un fallimento, perché dal punto di vista dell'utente la richiesta non ha prodotto quanto atteso. Le chiamate a `/health` sono invece **escluse** dal conteggio: sono interrogazioni della piattaforma, ripetute a intervalli fissi e quasi sempre coronate da successo, e includerle gonfierebbe l'indicatore con traffico che nessun utente ha generato.

L'obiettivo di throughput si aggancia allo scenario di scalabilità dell'Assignment 1, che chiede che un raddoppio del volume di ordini non produca degrado: dichiarare un livello sostenuto lo trasforma da previsione in impegno verificabile.

**Error budget.** Promettere il 99,5% significa ammettere che lo 0,5% può fallire. In 30 giorni ci sono 43.200 minuti, quindi quello 0,5% vale **216 minuti**: circa tre ore e mezza in cui il servizio può non funzionare senza aver mancato l'obiettivo.

### 3.2 Strumentazione

Entrambi gli indicatori si ricavano contando le richieste servite, distinguendo quelle riuscite dal totale. Le metriche sono esposte dall'api-gateway sulla porta 9492.

Tabella 6: Metriche introdotte nell'Assignment 3

| Metrica                                          | Tipo    | Contenuto                         |
|--------------------------------------------------|---------|-----------------------------------|
| <code>gateway_requests_total</code>              | counter | richieste servite                 |
| <code>gateway_successful_requests_total</code>   | counter | quelle con status inferiore a 400 |
| <code>gateway_response_time_seconds_total</code> | counter | tempo di risposta accumulato      |

I primi due counter sono quanto serve agli indicatori: il loro rapporto è la disponibilità, la derivata del primo è il throughput. Il terzo non alimenta alcun obiettivo, ma diviso per il numero di richieste fornisce il tempo medio di risposta, che resta un dato utile a osservare il sistema.

Nessuna metrica porta label. Si era valutato di distinguere le richieste per percorso, ma le rotte contengono identificatori di risorsa, e usarli come valori di label farebbe crescere senza limite il numero di serie temporali. La misura viene presa quando la risposta è stata scritta, unico momento in cui sia lo status code sia la durata sono definitivi.

Listing 8: Le due query PromQL degli indicatori

```
sum(increase(gateway_successful_requests_total[30d]))
/ sum(increase(gateway_requests_total[30d]))
```



```
sum(rate(gateway_requests_total[30d]))
```

Il gateway è replicato, quindi entrambe le query sommano su tutte le istanze: i counter sono per processo, e la somma li ricompone in un indicatore di sistema.

### 3.3 Verifica degli indicatori

Gli indicatori sono stati misurati sul deployment Kubernetes, sottoponendo il gateway a un carico generato dall'interno del cluster da dodici client concorrenti, ciascuno dei quali ripete un ciclo di due richieste valide e una destinata a fallire, per un totale di 8.100 richieste servite.

Tabella 7: Valori osservati

| Indicatore    | Atteso                              | Misurato           |
|---------------|-------------------------------------|--------------------|
| disponibilità | 0,667, ossia due successi su tre    | <b>0,6667</b>      |
| throughput    | superiore a 40 richieste al secondo | <b>138,5 req/s</b> |

La coincidenza fra valore atteso e valore misurato per il primo indicatore è la verifica che la metrica conteggia ciò che dichiara di conteggiare. Il secondo è il picco sostenuto durante la prova, distribuito in modo uniforme sulle tre repliche, che hanno servito rispettivamente 2.733, 2.708 e 2.659 richieste.

Una prova precedente, condotta con un solo client sequenziale, aveva registrato un picco di appena 15,4 richieste al secondo. Il limite non era del sistema ma del generatore: un client che attende ogni risposta prima di inviare la successiva non può produrre più richieste di quante ne completi in sequenza.

Il protocollo di misura non è arbitrario: il **modello pull** di Prometheus impone tre condizioni, in assenza delle quali l'indicatore restituisce un valore plausibile e falso.

**Il carico deve durare più di un intervallo di scrape.** Prometheus non osserva le richieste una per una: passa a intervalli regolari e legge il valore dei contatori. Se tutte le richieste arrivano fra un passaggio e il successivo, alla prima lettura il contatore è già al valore finale e da lì in poi non cresce più. La funzione `rate()`, che calcola di quanto un contatore è cresciuto, non può usare quella prima lettura, perché non sa da quale valore partisse: vede solo la parte costante che segue e risponde zero, anche se nel frattempo erano fallite decine di richieste.

**Il carico va generato dall'interno del cluster.** Raggiungere il gateway da fuori con `kubectl port-forward` sembra comodo, ma quel comando sceglie un pod all'inizio e continua a usare sempre quello: tutte le richieste finirebbero su una replica sola, e le altre due risulterebbero inattive. Inviando invece il carico dall'interno del cluster, all'indirizzo del Service, il bilanciamento avviene davvero, come mostrano i conteggi quasi identici riportati sopra.

### 3.4 Deployment su Kubernetes

Il sistema è descritto da sette manifest nella cartella `kubernetes/`: un namespace dedicato, i quattro servizi, l'infrastruttura Kafka con Zookeeper, e Prometheus. Il cluster

locale utilizzato è **kind**. Poiché le immagini sono costruite localmente e non pubblicate su un registry, vanno caricate nel nodo con `kind load docker-image` e i Deployment specificano `imagePullPolicy: IfNotPresent`.

L'osservazione più rilevante riguarda ciò che **non** è stato necessario fare. Le variabili d'ambiente dei manifest sono identiche a quelle del Docker Compose, perché i servizi si individuano per nome di Service e il DNS interno di Kubernetes li risolve esattamente come faceva quello di Docker. Il pattern **Externalized Configuration**, adottato nell'Assignment 2, ha quindi reso il passaggio di piattaforma possibile **senza modificare una riga di codice Java**.

Oltre a Deployment e Service, i manifest aggiungono due elementi non strettamente necessari a far funzionare il sistema. Il primo sono le **probe** di liveness e readiness su tutti e quattro i servizi, possibili senza scrivere codice perché l'endpoint `/health` esisteva già dall'Assignment 2. Il secondo è **Prometheus**, deployato nel cluster insieme ai servizi, senza il quale gli indicatori sarebbero misurabili solo in esecuzione locale.

La distinzione fra le due probe merita di essere esplicitata, perché agiscono su piani diversi: la *liveness* accerta che il processo sia vivo e in caso contrario lo sostituisce, mentre la *readiness* accerta che sia pronto a servire e in caso contrario lo rimuove dagli endpoint del Service senza ucciderlo. Un servizio che sta ancora sottoscrivendo i propri canali è vivo ma non pronto.

### 3.5 Replicazione

Il numero di repliche è portato a **tre solo per l'api-gateway**, che è l'unico componente privo di stato proprio. Gli altri restano a una replica perché il loro stato risiede in memoria: con più repliche il drone-service assegnerebbe lo stesso drone a due consegne diverse. È il limite già dichiarato nel report dell'Assignment 2, che il presente lavoro non rimuove ma circoscrive.

Va precisato che cosa questa replicazione produce, per non attribuirle più di quanto dia. Produce **disponibilità**: la caduta di un'istanza non interrompe il servizio, come mostrato nella sezione successiva. Non produce **throughput**, per tre ragioni concorrenti. Il collo di bottiglia resta a valle, perché ogni operazione di dominio converge su un delivery-service a replica singola. Il canale di risposta è in broadcast, quindi il lavoro di lettura da Kafka viene moltiplicato per il numero di repliche anziché diviso. E ogni richiesta in volo trattiene un thread fino alla risposta o al timeout. Aumentare la capacità del sistema richiederebbe prima di estrarre lo stato del delivery-service, poi di aumentare le partizioni dei suoi canali.

Replicare il gateway impone due vincoli che con una sola istanza non si porrebbero, e che hanno determinato altrettante scelte di progetto.

**Un consumer group per istanza, non per servizio.** All'interno di un gruppo ogni partizione appartiene a un solo membro, e i canali di risposta ne hanno una soltanto: se le tre repliche condividessero il gruppo **api-gateway**, tutte le risposte verrebbero recapitate a una sola di esse e le altre due non ne riceverebbero alcuna. Poiché però il Service distribuisce le richieste in ingresso su tutte e tre, la replica che attende una risposta coincide con quella che la riceve soltanto in un caso su tre: le altre due volte l'attesa si chiuderebbe in timeout.

La risposta a un `requestId` appartiene infatti alla replica che ne custodisce l'attesa in memoria: il canale di risposta è un **broadcast**, non un carico da distribuire. Il gruppo

è quindi reso univoco per istanza, così che ogni replica riceva tutte le risposte e scarti quelle non proprie. Gli altri tre servizi mantengono il gruppo fisso, che per essi è corretto perché le loro reazioni sono autosufficienti.

Listing 9: Un gruppo per istanza, non per servizio

```
static final String CONSUMER_GROUP = "api-gateway-" + UUID.  
    randomUUID();
```

**Un Service headless per lo scrape delle metriche.** Ogni replica tiene i propri contatori in memoria, quindi i tre valori sono diversi fra loro. Un Service ordinario espone però un solo indirizzo e a ogni connessione ne sceglie una: Prometheus finirebbe per leggere ora l'una ora l'altra, credendo di interrogare sempre la stessa sorgente. Il valore che registrerebbe salirebbe e scenderebbe senza motivo apparente, e poiché un contatore che diminuisce viene interpretato come un riavvio del processo, i tassi calcolati su quella serie sarebbero privi di senso.

Serve quindi che Prometheus veda i tre pod come tre sorgenti distinte. Lo si ottiene con un secondo Service dichiarato **headless**, cioè privo di indirizzo proprio: il suo nome non corrisponde a un punto d'ingresso unico, ma si risolve nell'elenco degli indirizzi di tutti i pod. Prometheus, configurato per ricavare le proprie destinazioni da quella risoluzione, ne ottiene tre e le interroga separatamente. La verifica sul cluster mostra **sei target attivi, di cui tre distinti per il gateway**, uno per replica, e le query degli indicatori li ricompongono sommando.

Esiste un'alternativa in cui Prometheus interroga direttamente Kubernetes per farsi elencare i pod, più generale perché non richiede un Service dedicato, ma comporta l'introduzione di tre oggetti di controllo degli accessi che il resto del sistema non usa.

### 3.6 Self-healing osservato

Il beneficio più immediato della piattaforma è il control loop che riporta lo stato osservato a quello dichiarato. Eliminando manualmente un pod dell'api-gateway mentre il sistema era sotto carico, il ReplicaSet ne ha creato immediatamente un altro, divenuto pronto in **45 secondi**, mentre le due repliche superstiti continuavano a servire senza interruzione: è la disponibilità che la replicazione promette, verificata invece che asserita.

Il broker è l'unico componente privo di probe, a differenza dei quattro servizi: non espone un endpoint HTTP di salute, e i controlli di tipo **exec** disponibili si sono rivelati inadatti a descriverne la disponibilità. La sua indisponibilità temporanea non blocca però il sistema, perché i servizi che vi si collegano gestiscono da sé la riconnessione: la loro partenza non dipende dall'ordine di avvio, come si è osservato durante le prove, quando un riavvio del broker non ha richiesto di riavviare nulla d'altro.

A verifica conclusa, i pod applicativi hanno mantenuto lo stato pronto per **18 ore consecutive senza alcun riavvio**.

## 4 Impatto sui test

Il passaggio a un flusso non sincrono ha effetti diretti sulla verifica, perché i test dell'Assignment 2 asserivano immediatamente dopo l'azione, assumendo che al ritorno della chiamata l'intera catena fosse conclusa. Tale assunzione non vale più per costruzione.

I test sono stati quindi riformulati in modo da **attendere che il sistema si assesti**, interrogando ripetutamente entro un budget di tempo anziché verificare una sola volta. Lo scenario end-to-end esprime ora questa attesa nel proprio testo, con la formulazione *the delivery eventually has a drone assigned*.

È stata inoltre introdotta la classe `BrokerAvailability`, che sospende i test dipendenti dal middleware, tramite `Assumptions.abort()`, quando il broker non è raggiungibile: un test che non può essere eseguito deve dichiararsi saltato e non fallito, altrimenti l'esito perde significato.

## 5 Limiti noti

**Il delivery-service non è replicabile, e il motivo è duplice.** Event store, snapshot e read model restano strutture dati in memoria, quindi due repliche avrebbero due verità diverse. A questo si aggiunge un secondo ostacolo, meno ovvio e specifico dell'architettura a eventi: i sette canali in ingresso del servizio sono sottoscritti sotto un unico consumer group, per cui Kafka distribuisce le loro partizioni **fra** le repliche invece di replicarne il consumo.

La cosa è stata verificata sul cluster portando il servizio a due repliche. L'assegnazione osservata è stata la seguente:

Listing 10: Assegnazione dei canali con due repliche del delivery-service

```
pod A    order-confirmed, drone-unavailable
pod B    drone-assigned, drone-position-reports, get-delivery-
         requests
```

Il ciclo di vita di una singola consegna viene così spezzato fra due processi con memorie separate: la consegna nasce in A, ma l'assegnazione del drone arriva in B, che non la conosce. Il comportamento osservato è stato che nessuna consegna riceve più un drone e resta indefinitamente in stato `SCHEDULED`, con il seguente messaggio nel log della replica B:

Listing 11: Esito osservato

```
WARNING: DroneAssigned discarded - Delivery not found: c5c7fdb
e-565e-4e8a-b211-a6ff4d86508d
```

Va sottolineato che il guasto non è dovuto a Kafka, che si è comportato correttamente, né all'architettura a eventi: la stessa configurazione con lo stato in memoria si sarebbe rotta identicamente nella versione sincrona dell'Assignment 2. Rendere il servizio replicabile richiede, nell'ordine, uno store condiviso, un numero di partizioni maggiore di uno, e solo allora l'aumento delle repliche.

**Salvare gli eventi su file non basterebbe.** È l'evoluzione più immediata e richiederebbe di sostituire un solo adapter, senza toccare il dominio, ma darebbe **durabilità e non replicabilità**: due processi che scrivono lo stesso file non hanno modo di coordinarsi, e il controllo di concorrenza oggi garantito all'interno di una sola macchina virtuale verrebbe meno. Servirebbe uno store realmente condiviso, come un database, che imponga esso stesso il controllo di versione. Su Kubernetes, inoltre, il file va montato su un volume persistente, altrimenti sparisce a ogni riavvio del pod.

**Uno stesso evento può essere elaborato due volte.** Elaborare un evento e registrare di averlo fatto sono due operazioni distinte, e il segnalibro del consumatore avanza a intervalli anziché dopo ogni messaggio: un processo che termini fra le due riprende dall'evento che aveva già elaborato. È la garanzia *at-least-once*, e Kafka la preferisce all'alternativa, che sarebbe perdere l'evento anziché ripeterlo.

Gli handler dovrebbero quindi essere **idempotenti**, cioè produrre lo stesso risultato se eseguiti una o più volte, e i nostri non lo sono: rileggere `order-confirmed` creerebbe una seconda consegna per lo stesso ordine, e impegnerebbe un secondo drone. Sarebbe sufficiente verificare l'esistenza di una consegna per quell'ordine prima di crearne una, interrogazione a cui il read model sa già rispondere.

**La finestra di 30 giorni non è osservabile.** Gli obiettivi sono enunciati su 30 giorni perché è la scala su cui un service level ha senso, ma il sistema viene avviato per la durata delle prove e Prometheus non possiede quella storia. Le verifiche usano una finestra di pochi minuti: cambia l'ampiezza del campione, non la forma dell'indicatore.

## 6 Conclusioni

Sul piano della comunicazione, l'architettura a eventi ha sostituito l'**orchestrazione** con la **coreografia**. Il risultato più sintetico è la firma del costruttore dell'application service, passata da tre dipendenze a una: il delivery-service non conosce più l'esistenza degli altri servizi e si limita a produrre fatti. Il prezzo è una consistenza *eventual*, che rende normale uno stato intermedio prima considerato degradato.

Sul piano operativo, i service level hanno reso continua una verifica che prima avveniva una volta sola: gli scenari di qualità si controllavano provocando una condizione, gli obiettivi si osservano invece senza interruzione. Resta valido il principio enunciato nell'Assignment 2, **lo strumento di misura determina quali attributi di qualità è possibile promettere**: la latenza continua a non essere osservabile, e finché non lo sarà nessun obiettivo potrà essere formulato su di essa.

Le due direzioni si incontrano nel deployment. Replicare un componente ha fatto emergere due difetti che con una sola istanza non si sarebbero mai visti, il consumer group condiviso e lo scrape attraverso un servizio bilanciato: una scelta di deployment non è neutrale rispetto all'architettura applicativa, ma ne verifica le assunzioni.